Roma Tre

Università degli Studi

Dipartimento di Ingegneria Civile, Informatica
e delle Tecnologie Aeronautiche

Tesi di Laurea

Protecting the autonomic manager

A comparative analysis of Quorum and CometBFT as blockchain backends for Self-Protecting Systems

Corso di Laurea: Ingegneria Informatica

Candidato: Valerio Tatti

Matricola: 609824

Relatore: Prof. Stefano Iannucci

Anno Accademico: 2025/2026

Contents

1 Introduction **1**

1.1 The Paradigm of Self-Protecting Systems 1
1.2 The Vulnerability of the Autonomic Manager 2
1.3 Blockchain as a Mechanism of Distributed Trust 2
1.4 Objectives and Contributions of the Thesis 3
1.5 Structure of the Thesis 4

2 Technologies Setting and Related Work **5**

2.1 Autonomic Computing and the MAPE Cycle 5
2.1.1 Hystorical notes on the complexity crysis and its overcoming 5
2.1.2 The Four Aspects of Self-Management 5
2.1.3 The Structure of Autonomic Elements 6
2.1.4 Multi-Agent Dynamics and Security Challenges 7
2.2 Blockchain in Permissioned Environments 7
2.3 Distributed Consensus Algorithms 7
2.4 State of the Art in Protecting the Manager 7

3 A Cyber-Resilient Architectural Model **8**

3.1 Architecture Overview 8
3.2 State Representation and Response Logic 8
3.3 Distributed Voting Mechanism 8
3.4 Threat Model 8

4 Requirement Alignment: Quorum vs. CometBFT **9**

4.1 Specific Requirements for SPS Applications 9
4.2 Analysis of GoQuorum 9
4.3 Analysis of CometBFT 9
4.4 Architectural Comparison 9

5 Implementation details **10**

5.1 Experimental Setup and Network Topology 10
5.2 Rust vs Solidity middleware 10
5.3 Alert detection and response loop 10

5.4 Blockchain config generation 10

6 Experiments and results **11**
6.1 Evaluation Methodology 11
6.2 Performance Analysis 11
6.2.1 Effectiveness of the Deduplication Mechanism 11
6.2.2 End-to-End Latency under Low Traffic 11
6.2.3 Transaction Throughput under Increasing Load 11

7 Conclusions and Future Work **12**
7.1 Summary of Research Findings 12
7.2 Limitations of the Study 12
7.3 Future Directions 12

List of Figures

List of Tables

Chapter 1

Introduction

In modern IT infrastructures, the rapid shift toward cloud-native, containerized microservices has dramatically expanded the digital attack surface. The escalation of sophisticated, high-velocity cyber threats and the ever-growing volume of sensitive data housed in these environments have driven the need for computing systems capable of defending themselves autonomously. Within this context, the autonomic computing paradigm has become a cornerstone of modern security engineering, aiming to reduce human operational burden and accelerate threat response. However, while automation resolves many latency and coordination challenges, it introduces new systemic vulnerabilities. If the automated security logic itself is targeted, it can become a vector for devastating privilege escalation and system-wide compromise.

1.1 The Paradigm of Self-Protecting Systems

Self-Protecting Systems [1] (SPS) are autonomic software systems designed to detect, analyze, and mitigate security threats dynamically with limited or no human intervention. Following the classic autonomic feedback loop, an SPS typically comprises two primary macro-components: a *managed system* and an *autonomic manager*. The managed system represents the production environment, services, or software applications exposed to potential attacks. Conversely, the autonomic manager serves as the central controller responsible for executing the security loop.

To achieve continuous protection, the autonomic manager organizes its operations into a pipeline governed by two main functional sub-components:

- **Intrusion Detection Systems (IDS):** Passive or active security sensors distributed across the managed system to monitor traffic, logs, or system calls and detect anomalous behavior or known attack signatures.
- **Intrusion Response Systems (IRS):** Decision-making engines responsible for selecting and executing the appropriate response actions (countermeasures)

to neutralize or mitigate the detected threats.

1.2 The Vulnerability of the Autonomic Manager

Although highly effective in mitigating threats directed at the managed system, this canonical design suffers from a critical structural vulnerability: the centralization of the autonomic manager. An adversary who penetrates the infrastructure can target the autonomic manager itself. By compromising this central controller, the attacker can silence intrusion alerts, disable response mechanisms, or even manipulate the manager into executing malicious countermeasures (e.g., disconnecting legitimate network segments or isolating critical database services).

Indeed, protecting the controller that protects the system remains an open and central challenge in autonomic security research. If the security logic is centralized and compromised, the integrity of the entire infrastructure is undermined, rendering the self-protecting features not only ineffective, but actively weaponized against the system.

1.3 Blockchain as a Mechanism of Distributed Trust

To eliminate the single point of failure represented by a centralized manager, recent state-of-the-art research [2] has proposed distributing the autonomic manager across a permissioned blockchain network. In this decentralized architecture, blockchain technology acts as a tamper-proof ledger and a distributed state machine (DSM) that logs security events and executes response logic securely through smart contracts.

This distributed approach establishes fundamental security guarantees that collectively safeguard the autonomic loop. To prevent a single compromised IDS sensor from injecting false alerts and triggering unauthorized countermeasures, the smart contract enforces a consensual voting mechanism. Under this scheme, updates to the shared security state are only committed once they receive corroborating reports from multiple independent, heterogeneous IDSs, ensuring that isolated sensor compromises do not lead to malicious actuations.

Also, by deploying the autonomic manager across a network of nodes running a Byzantine Fault-Tolerant (BFT) consensus protocol, the security loop gains strong structural resilience. This consensus mechanism allows the system to withstand the active compromise of up to 1/3 of the participating nodes—encompassing both IDSs and validators—while strictly maintaining both the availability and the integrity of the underlying ledger under partial network compromise.

Generally, the system is designed to reduce the attack surface as much as

possible treating actuators as black boxes (blind), the actuators receive signed mitigation commands directly from the blockchain state, rather than querying the state of potentially compromised application containers directly.

1.4 Objectives and Contributions of the Thesis

While the theoretical integration of blockchain into the SPS architecture provides outstanding security guarantees, the practical choice of the underlying blockchain technology is not neutral. A blockchain-based autonomic manager has unique requirements: it must handle sparse, highly event-driven alert traffic with exceptionally low latency and high determinism. Conversely, many features native to public blockchains (such as virtual-machine-level gas metering, token economics, transaction fees, and dynamic peer discovery) add unnecessary complexity, expand the trusted computing base (TCB), and increase the attack surface.

The natural baseline choice for permissioned enterprise setups is GoQuorum (a fork of Ethereum). However, this thesis argues that general-purpose smart contract platforms incur severe, non-intrinsic performance overheads due to interpreted virtual machine (EVM) bytecode execution, rigid transaction fee mechanisms, and fixed-interval block production. As an alternative, this work explores CometBFT (the successor of Tendermint), a lower-level BFT consensus engine that supports highly optimized, application-specific distributed state machines.

The primary objective of this thesis is to present a comprehensive qualitative and empirical comparison between a conventional general-purpose platform (GoQuorum) and a requirement-driven, application-specific alternative (CometBFT) to determine which architecture best supports the performance and cyber-resilience demands of modern Self-Protecting Systems.

Specifically, the core contributions of this thesis are as follows:

1. **Requirement Formulation:** We define and classify a comprehensive set of functional and non-functional requirements (required, desirable, and not needed) for blockchain technology in the specific context of an autonomic SPS loop.

2. **Architectural Analysis:** We conduct a thorough qualitative comparison of GoQuorum and CometBFT, highlighting the architectural mismatch of general-purpose virtual machines versus application-specific state machines.

3. **Prototype Implementation:** We design and implement a realistic, fully containerized SPS prototype emulated via Kathará on top of Docker. The prototype integrates three heterogeneous IDS engines (Snort, Suricata, and Zeek) protecting a vulnerable web application (OWASP Juice Shop).

4. **Deduplication Middleware:** We propose and develop a custom deduplication middleware layer for the member nodes. This layer filters repetitive alerts, preventing mempool saturation on the ledger keeping transaction number upper limited per block.

5. **Empirical Evaluation:** We perform extensive automated benchmarks under controlled attack conditions, comparing GoQuorum and CometBFT across key performance indicators, including end-to-end response latency, transaction throughput under heavy load, and CPU resource utilization.

1.5 Structure of the Thesis

To thoroughly explore the theoretical background, architectural design, and empirical evaluation of the proposed systems, the remainder of this thesis is structured as follows:

Chapter 2 – Technologies Setting and Related Work: Introduces the foundational concepts of autonomic computing, the MAPE cycle, Byzantine fault-tolerant consensus, the structural differences between public and permissioned blockchain systems, and reviews the state of the art in autonomic security.

Chapter 3 – A Cyber-Resilient Architectural Model: Details the technology-agnostic architecture of the Self-Protecting System, focusing on the interactions between IDSs, the smart contract state, and the blind actuators.

Chapter 4 – Requirement Alignment: Quorum vs. CometBFT: Formally identifies the blockchain properties strictly required for SPS applications and theoretically compares Quorum's general-purpose execution environment with CometBFT's application-specific design.

Chapter 5 – Implementation Details: Describes the prototype implementation, the containerized network topology emulated via Kathará, and the custom middleware developed for the evaluation.

Chapter 6 – Experiments and Results: Analyzes the performance of the two blockchain backends under varying workloads, presenting metrics on end-to-end latency, transaction throughput, and resource consumption.

Chapter 7 – Conclusions and Future Work: Summarizes the findings of the comparative analysis, acknowledges current limitations, and outlines directions for future research in decentralized system protection.

Chapter 2

Technologies Setting and Related Work

2.1 Autonomic Computing and the MAPE Cycle

2.1.1 Hystorical notes on the complexity crysis and its overcoming

The paradigm of Autonomic Computing formally emerged in October 2001, when IBM released a manifesto identifying an upcoming software complexity crisis as the primary obstacle to further progress in the IT industry[cite: 918, 928]. As computing environments began to weigh in at millions of lines of code, the expertise required for installation, configuration, and maintenance approached a limit on human capability to train new professionals. This challenge is exacerbated by the necessity of integrating several heterogeneous environments into corporate-wide systems and extending them globally across the Internet[cite: 930, 931]. IBM observed that relying solely on programming methods innovations would not suffice to manage such massive and complex systems, which often require timely responses to rapid streams of changing demands[cite: 934, 936, 937].

To address this problem IBM proposed the vision of autonomic computing: systems capable of managing themselves based on high-level objectives from human administrators[cite: 939, 940]. The term "autonomic" was chosen for its biological connotation, tracing an analogy to the human autonomic nervous system[cite: 940, 941]. Just as the biological system governs vital, low-level functions such as heart rate and body temperature without conscious effort, autonomic computing aims to free human administrators from the burden of managing low-level system operations through automation[cite: 941, 969].

2.1.2 The Four Aspects of Self-Management

The essence of autonomic computing is self-management, intended to provide software that can adjust it's behaviour in accordance to human's objectives without

their intervention[cite: 968, 969]. IBM characterizes self-management through four fundamental aspects, often referred to as the "self-*" properties[cite: 954, 983]:

- **Self-configuration**: Systems must automatically configure components and integrate themselves into complex environments following high-level policies[cite: 984]. New components should be able to incorporate seamlessly treating other components as black boxes (history proved them right, Docker is now the de-facto standard in the deploying of application).
- **Self-optimization**: Autonomic systems continually seek opportunities to improve their own performance and efficiency[cite: 984, 993]. In environments like complex middleware or databases that possess hundreds of manually set parameters, the system must monitor, experiment with, and tune these parameters dynamically[cite: 990, 994] (Again, AWS Auto-scaling is an example of self-optimization).
- **Self-healing**: The system is designed to automatically detect, diagnose, and repair localized software and hardware problems[cite: 984, 1000]. Using knowledge of the system configuration and logs, the autonomic manager can identify root causes of failures, match them against known patches, and retest the system without manual debugging[cite: 1000, 1001] (Kubernetes's Liveness probe is another industry standard).
- **Self-protection**: Autonomic systems defend the infrastructure as a whole against malicious attacks[cite: 984, 1004]. They use sensor reports to anticipate problems and take proactive steps to avoid or mitigate systemwide failures[cite: 1005](Cloudflare's Web application firewall).

2.1.3 The Structure of Autonomic Elements

Architecturally, autonomic systems are composed by elements that manage their own internal behavior and relationships with others in accordance with established policies[cite: 1008]. An autonomic element typically consists of one or more managed elements coupled with a single autonomic manager[cite: 1022].

- **Managed Element**: This component can be found in non-autonomic systems and can be an hardware resource (such as storage or a CPU) or a software resource (such as a database or a web service).
- **Autonomic Manager**: The manager distinguishes the autonomic element by monitoring the managed element and its environment, and executing action based on the analysis of their information.

2.1.4 Multi-Agent Dynamics and Security Challenges

Autonomic elements unlike typical Web services, do not honor requests unconditionally; they provide services only if doing so is consistent with their internal goals[cite: 1047, 1048].

This high degree of autonomy introduces critical system-level issues that can't be ignored when looking for attack surfaces, because these systems rely on high-level goals, they are vulnerable to new forms of attack where an adversary might manipulate policies to gain systemic leverage[cite: 1144, 1145]. This highlights the necessity for a secure-by-design management infrastructure that can address the "Who watches the watchers?" situation of autonomic elements[cite: 1146, 1169, 1172].

2.2 Blockchain in Permissioned Environments

2.3 Distributed Consensus Algorithms

2.4 State of the Art in Protecting the Manager

Chapter 3

A Cyber-Resilient Architectural Model

3.1 Architecture Overview

3.2 State Representation and Response Logic

3.3 Distributed Voting Mechanism

3.4 Threat Model

Chapter 4

Requirement Alignment: Quorum vs. CometBFT

4.1 Specific Requirements for SPS Applications

4.2 Analysis of GoQuorum

4.3 Analysis of CometBFT

4.4 Architectural Comparison

Chapter 5

Implementation details

5.1 Experimental Setup and Network Topology

5.2 Rust vs Solidity middleware

5.3 Alert detection and response loop

5.4 Blockchain config generation

Chapter 6

Experiments and results

6.1 Evaluation Methodology

6.2 Performance Analysis

6.2.1 Effectiveness of the Deduplication Mechanism

6.2.2 End-to-End Latency under Low Traffic

6.2.3 Transaction Throughput under Increasing Load

Chapter 7

Conclusions and Future Work

7.1 Summary of Research Findings

7.2 Limitations of the Study

7.3 Future Directions

Bibliography

[1] Eric Yuan, Naeem Esfahani, and Sam Malek. "A Systematic Survey of Self-Protecting Software Systems". In: *ACM Trans. Auton. Adapt. Syst.* 8.4 (Jan. 2014). ISSN: 1556-4665. DOI: 10.1145/2555611. URL: https://doi.org/10.1145/2555611.

[2] Tommaso Caiazzi et al. "A Novel Architecture for Cyber-Resilient Self-Protecting Systems Based on Blockchain". In: *2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)*. 2025, pp. 1–8. DOI: 10.1109/COMPSAC65507.2025.00010.